CLUSTERIZAÇÃO

Guia Completo — Nível Especialista

Breno Ferrari · Preparação Claro · Abril 2026

#	Seção
1	Fundamentos e Definição do Problema
2	EDA para Clusterização
3	Variáveis Contínuas vs Categóricas
4	Pré-processamento
5	Redução de Dimensionalidade
6	Algoritmos — Conceito, Uso e Código
7	Seleção do Número de Clusters
8	Métricas de Validação
9	Visualizações e Análise dos Clusters
10	Pipeline Completo em Produção
11	Monitoramento e Retreinamento
12	Checklist do Especialista

1 · FUNDAMENTOS E DEFINIÇÃO DO PROBLEMA

Clusterização é aprendizado não supervisionado: não existe variável resposta Y. O objetivo é encontrar estrutura natural nos dados — grupos de observações mais similares entre si do que com o restante. A ausência de ground truth muda radicalmente como você formula, executa e valida o problema.

1.1 A Mudança de Frame Mental

- **Classificação**: você tem Y, o modelo aprende a separar classes conhecidas.
- **Clusterização**: não há Y. Os grupos emergem dos dados — você descobre, não confirma.
- **Avaliação**: não existe acurácia. A validação é matemática + visual + de negócio.
- **Subjetividade**: dois cientistas podem chegar a segmentações diferentes com os mesmos dados — ambas podem ser válidas se fizerem sentido de negócio.

1.2 Antes de Qualquer Código — As Perguntas Certas

Objetivo de negócio:

- O que será feito com os clusters? Campanhas, precificação, produto, atendimento?
- Qual é a unidade de análise? Cliente, contrato, transação, aparelho, endereço?
- Quantos grupos são operacionalmente viáveis? Marketing consegue trabalhar com 15 segmentos?
- Os clusters precisam ser estáticos ou dinâmicos (cliente pode migrar entre clusters)?
- Qual é o horizonte temporal? Segmentação mensal, trimestral, anual?

Definição das dimensões de análise:

Features devem representar dimensões do negócio, não apenas disponibilidade de dados. Em telecom, dimensões típicas para segmentação de clientes:

Dimensão	Features Típicas	Objetivo
Valor	ARPU, LTV, ticket médio, receita acumulada	Separar clientes por retorno financeiro
Engajamento	Consumo de dados, chamadas, canais acessados, frequência de uso	Separar por intensidade de uso
Risco	Churn score, reclamações, atrasos de pagamento	Separar por propensão à perda
Perfil	Tempo de contrato, tipo de plano, região, dispositivo	Segmentar por perfil demográfico/contratual
Comportamento	Horário de uso, sazonalidade, mix de serviços	Separar por padrão comportamental

Regra crítica: não inclua variáveis que são consequência do que você quer segmentar. Se o objetivo é segmentar por comportamento de uso, não inclua o plano contratado como feature — o plano é resultado do comportamento, não causa. Isso introduz circularidade.

2 · EDA PARA CLUSTERIZAÇÃO

EDA em clusterização tem um objetivo adicional além do padrão: entender a estrutura natural dos dados antes de impor qualquer algoritmo. Você quer saber se clusters existem — não apenas se as variáveis têm qualidade.

2.1 Análise Univariada das Features

- **Distribuição:** histograma e KDE de cada feature numérica. Features muito assimétricas podem exigir transformação log.
- **Escala:** identificar features com ranges muito diferentes (renda em R$ vs consumo em GB). Escala dominante distorce distâncias.

• **Outliers:** boxplot e Z-score. Outliers extremos puxam centroides no K-Means — tratar antes.

• **Nulos:** taxa por variável. Clusterização não tolera nulos — imputação obrigatória.

• **Variância:** features com variância zero ou quase zero não contribuem para separação — remover.

2.2 Análise de Estrutura dos Dados

Esse bloco é específico de clusterização — você quer evidências de que estrutura natural existe antes de escolher o algoritmo.

• **Hopkins Statistic:** testa se os dados têm tendência a clusterização. Valores próximos de 0.5 = distribuição aleatória (sem estrutura). Valores próximos de 1 = estrutura de cluster forte. Valor > 0.75 justifica clusterização.

• **Matriz de correlação:** features altamente correlacionadas entre si indicam redundância. Considerar PCA ou remover uma delas.

• **Pairplot:** scatter matrix de todas as features — visualmente identifica agrupamentos naturais.

• **Distância média entre pontos:** se todos os pontos estão equidistantes, há maldição da dimensionalidade.

Hopkins Statistic — Código:

```
from sklearn.neighbors import NearestNeighbors import numpy as np def hopkins(X, m=None): if m is None: m = int(0.1 * len(X)) d = X.shape[1] nn = NearestNeighbors(n_neighbors=1).fit(X) rand_pts = np.random.uniform(X.min(0), X.max(0), (m, d)) sample = X[np.random.choice(len(X), m, replace=False)] u = nn.kneighbors(rand_pts, return_distance=True)[0].sum() w = nn.kneighbors(sample, return_distance=True)[0].sum() return u / (u + w) # > 0.75: estrutura presente
```

2.3 Gráficos e Técnicas de EDA para Clusterização

Gráfico/Técnica	O que revela	Quando usar
Histograma + KDE	Distribuição e assimetria de cada feature	Sempre — primeiro passo
Boxplot	Outliers e IQR por feature	Sempre antes de escalar
Pairplot (Seaborn)	Estrutura visual entre pares de features	Até ~10 features
Heatmap de Correlação	Multicolinearidade e redundância	Sempre
Scatter Plot 2D	Agrupamentos visíveis em duas dimensões	Features mais relevantes par a par
Parallel Coordinates	Perfil multidimensional por grupo	Após definir clusters — validação
t-SNE / UMAP 2D	Estrutura de alta dimensão em 2D	Antes e após clusterização
Hopkins Statistic	Evidência matemática de estrutura	Antes de escolher algoritmo
Dendrograma	Hierarquia natural dos dados	Exploração inicial — hierárquico

3 · VARIÁVEIS CONTÍNUAS vs CATEGÓRICAS

A maioria dos algoritmos de clusterização foi projetada para dados numéricos contínuos. Variáveis categóricas exigem tratamento específico — ignorar isso produz clusters sem sentido.

3.1 O Problema da Distância com Categóricas

K-Means usa distância euclidiana — que não faz sentido para categorias. A distância entre "pré-pago" e "pós-pago" não é matematicamente definida sem transformação.

3.2 Abordagens para Variáveis Mistas

Opção 1: Encoding + K-Means

- **One-Hot Encoding:** para categóricas de baixa cardinalidade (< 6 categorias). Cria colunas binárias. Problema: contribuição desproporcional se há muitas categorias.
- **Frequency Encoding:** substitui categoria pela sua frequência na base. Preserva a dimensão sem explodir.
- **Target Encoding:** não aplicável em clusterização (sem Y). Evitar.
- **Label Encoding:** só para ordinais com ordem real (baixo=0, médio=1, alto=2). Nunca para nominais.

Problema do OHE em K-Means: com muitas dummies, as variáveis binárias dominam a distância euclidiana. Padronizar as dummies junto com as numéricas atenua, mas não elimina o problema.

Opção 2: Distâncias Especializadas — Gower Distance

Distância de Gower é a solução mais robusta para dados mistos. Calcula distância parcial por tipo de variável e combina numa distância final [0,1]:

- **Numéricas:** distância de Manhattan normalizada pelo range: |xi - xj| / range
- **Categóricas nominais:** 0 se igual, 1 se diferente (Dice)
- **Binárias:** Jaccard ou Dice
- **Ordinais:** rank normalizado
- **Distância final:** média ponderada das distâncias parciais

```
import gower dist_matrix = gower.gower_matrix(df) # matriz n x n de distâncias # Usar com K-Medoids ou Hierarchical (não K-Means — precisa de centroide) from sklearn_extra.cluster import KMedoids km = KMedoids(n_clusters=4, metric="precomputed").fit(dist_matrix)
```

Opção 3: K-Modes e K-Prototypes

- **K-Modes:** versão do K-Means para dados 100% categóricos. Usa moda ao invés de média e distância de Hamming (número de categorias diferentes).
- **K-Prototypes:** combinação de K-Means (numéricas) e K-Modes (categóricas). Melhor opção para dados verdadeiramente mistos sem encoding.

```
from kmodes.kprototypes import KPrototypes kp = KPrototypes(n_clusters=4, init="Cao", n_init=5) # categorical: índices das colunas categóricas labels = kp.fit_predict(df.values, categorical=[2, 3, 5])
```

Opção 4: FAMD — Factor Analysis for Mixed Data

Técnica de redução de dimensionalidade para dados mistos. Combina PCA (numéricas) e MCA — Multiple Correspondence Analysis (categóricas) num espaço comum. Depois clusteriza nos componentes FAMD.

```
import prince famd = prince.FAMD(n_components=5) X_famd = famd.fit_transform(df) # df com mix de tipos # Clusterizar em X_famd com K-Means normalmente
```

Cenário	Abordagem Recomendada
Todas numéricas	StandardScaler + K-Means

Todas categóricas	K-Modes
Misto — poucas categóricas, baixa cardinalidade	OHE + StandardScaler + K-Means
Misto — muitas categóricas ou alta cardinalidade	Gower Distance + K-Medoids
Misto — melhor qualidade	K-Prototypes ou FAMD + K-Means
Alta dimensionalidade mista	FAMD + K-Means

4 · PRÉ-PROCESSAMENTO

Clusterização é mais sensível a pré-processamento do que qualquer modelo supervisionado. Features mal escaladas, outliers não tratados ou nulos ignorados produzem clusters que refletem artefatos dos dados, não estrutura real.

4.1 Tratamento de Nulos

- **Regra absoluta:** nenhum algoritmo padrão de clusterização tolera nulos. Imputação é obrigatória.
- **MCAR:** média (numéricas) ou moda (categóricas) — seguro.
- **MAR:** KNN Imputer ou Iterative Imputer — preserva correlação entre features.
- **MNAR:** criar flag binária de nulo antes de imputar — o padrão de ausência pode ser informativo para o cluster.
- **> 30% nulos:** avaliar se a feature deve ser mantida ou removida.

4.2 Tratamento de Outliers

Por que importa: K-Means usa média como centroide. Um cliente com ARPU de R$ 50.000 em uma base onde a média é R$ 80 puxa um centroide inteiro para si, criando um "cluster de um".

- **Capping (Winsorization):** substituir valores acima do P99 (ou P95) pelo valor do percentil. Padrão mais usado.
- **Remoção:** apenas se o outlier é erro comprovado de dado.
- **Cluster separado:** em algumas aplicações, o outlier IS o segmento (ex: cliente VIP com gasto extremo). Identifique antes e decida conscientemente.
- **Isolation Forest:** detectar outliers multivariados antes de clusterizar.

```
# Capping no P1 e P99 for col in num_cols: p1, p99 = df[col].quantile([0.01, 0.99]) df[col] = df[col].clip(lower=p1, upper=p99)
```

4.3 Scaling — Escala é Tudo em Clusterização

Método	Fórmula	Quando usar	Limitação
StandardScaler	(X - media) / std	Padrão — maioria dos casos	Sensível a outliers extremos
MinMaxScaler	(X - min) / (max - min)	Quando quer [0,1] explícito	Muito sensível a outliers
RobustScaler	(X - mediana) / IQR	Quando há outliers que não foram removidos	Menos intuitivo
MaxAbsScaler	X / max(|X|)	Dados esparsos	Mantém zero como zero

Regra prática: sempre usar StandardScaler como padrão. Trocar para RobustScaler se houver outliers que não puderam ser removidos.

```
from sklearn.preprocessing import StandardScaler scaler = StandardScaler() X_scaled = scaler.fit_transform(X_train) # fit apenas no treino X_test_scaled = scaler.transform(X_test) # transform no teste
```

NUNCA fazer fit_transform no dataset inteiro antes de separar amostras de validação. Isso vaza estatísticas do conjunto de validação para o treino.

4.4 Feature Selection para Clusterização

- **Remover variância zero:** features constantes não contribuem — usar VarianceThreshold.
- **Remover quasi-constantes:** features com > 99% do mesmo valor.
- **Remover altamente correlacionadas:** correlação > 0.9 entre duas features = redundância. Manter uma.
- **Evitar features de ID:** CPF, ID, nome — únicos por definição, destroem clustering.

• **Cuidado com proxies de Y:** em churn clustering, não incluir churn_score como feature — é o que você quer entender, não uma causa.

```
from sklearn.feature_selection import VarianceThreshold sel = VarianceThreshold(threshold=0.01)
X_sel = sel.fit_transform(X_scaled)
```

5 · REDUÇÃO DE DIMENSIONALIDADE

Com muitas features, dois problemas surgem que destroem a clusterização: maldição da dimensionalidade e dificuldade de interpretação. Redução de dimensionalidade resolve ambos.

5.1 A Maldição da Dimensionalidade

Em alta dimensão, todos os pontos tendem a ficar aproximadamente equidistantes entre si. Distância — o coração de K-Means, K-Medoids e Hierárquico — perde significado discriminativo. O resultado: clusters aleatórios que não refletem estrutura real.

> **Quando aplicar redução:** geralmente acima de 15-20 features numéricas. Com dados mistos, o threshold é menor — mais tipos = mais ruído.

5.2 PCA — Principal Component Analysis

PCA transforma features originais em componentes ortogonais (perpendiculares entre si) ordenados pela variância que explicam. O primeiro componente captura a maior variância, o segundo a maior variância restante ortogonal ao primeiro, e assim por diante.

- **Premissa:** features numéricas padronizadas. PCA é sensível à escala.
- **Output:** componentes principais — combinações lineares das features originais.
- **Como escolher n_components:** gráfico de variância acumulada (scree plot). Escolher número que explica 80-90% da variância.
- **Interpretabilidade:** os componentes não têm nome direto. Para explicar clusters, volte às features originais após definir os grupos.

```
from sklearn.decomposition import PCA import matplotlib.pyplot as plt # Scree plot para escolher n_components pca_full = PCA().fit(X_scaled) var_cum = pca_full.explained_variance_ratio_.cumsum() # Escolher n onde var_cum >= 0.85 n_comp = (var_cum >= 0.85).argmax() + 1 pca = PCA(n_components=n_comp) X_pca = pca.fit_transform(X_scaled) print(f"Componentes: {n_comp}, Variancia: {var_cum[n_comp-1]:.2%}")
```

5.3 t-SNE — t-Distributed Stochastic Neighbor Embedding

t-SNE é uma técnica de visualização não-linear. Preserva a estrutura local dos dados — pontos próximos no espaço original ficam próximos na projeção 2D. NÃO serve para clusterização — apenas visualização.

- **Uso correto:** visualizar se clusters existem antes de escolher K, e validar clusters após definí-los.
- **Parâmetro perplexity:** balanceia estrutura local vs global. Valores típicos: 5-50. Testar múltiplos.
- **Não determinístico:** resultados mudam a cada execução — fixar random_state.
- **Lento:** O(n^2) — inviável para > 100.000 linhas. Usar amostra.

```
from sklearn.manifold import TSNE tsne = TSNE(n_components=2, perplexity=30, random_state=42, n_iter=1000) X_tsne = tsne.fit_transform(X_scaled) # usar amostra se n > 50k
```

5.4 UMAP — Uniform Manifold Approximation and Projection

UMAP é mais rápido que t-SNE e preserva melhor a estrutura global. Pode ser usado tanto para visualização quanto como pré-processamento para clusterização — ao contrário do t-SNE.

- **n_neighbors:** controla balanço local/global. Menor = mais local (15-30 típico).
- **min_dist:** quão compactos são os clusters na projeção. 0.0-0.5 típico.
- **Vantagem sobre t-SNE:** mais rápido, preserva estrutura global, resultado mais estável.

```
import umap reducer = umap.UMAP(n_components=2, n_neighbors=15, min_dist=0.1, random_state=42) X_umap = reducer.fit_transform(X_scaled)
```

Técnica	Tipo	Uso em Clustering	Limitação

PCA	Linear	Pré-processamento antes de clusterizar	Só relações lineares
t-SNE	Não-linear	Visualização APENAS — não clusterizar	Lento, não-determinístico
UMAP	Não-linear	Visualização + pré-processamento	Parâmetros sensíveis
FAMD	Misto	Dados com numéricas + categóricas	Requer library prince
MCA	Categórico	Dados 100% categóricos	Menos intuitivo
Autoencoder	Não-linear profundo	Alta dimensão com padrões complexos	Requer Deep Learning

6 · ALGORITMOS — CONCEITO, USO E CÓDIGO

6.1 K-Means

Conceito: Particiona n observações em K clusters minimizando a inércia (soma das distâncias quadráticas de cada ponto ao centroide do seu cluster). Iterativo: atribui pontos ao centroide mais próximo, recalcula centroides, repete até convergência.

- **Distância usada:** Euclidiana — sensível à escala.
- **Centroide:** média aritmética dos pontos do cluster — sensível a outliers.
- **Convergência:** garantida, mas para no mínimo local — múltiplas inicializações são essenciais.
- **K-Means++:** inicialização inteligente que escolhe centroides iniciais distantes entre si. Reduz risco de mínimos locais ruins.

Quando usar:

- Ponto de partida e baseline para qualquer problema de segmentação
- Dados numéricos padronizados sem outliers extremos
- Quando você precisa de algo rápido, escalável e interpretável
- Clusters aproximadamente esféricos e de tamanho similar

Quando NÃO usar:

- Dados com outliers não tratados
- Clusters de formas irregulares ou densidades muito diferentes
- Dados categóricos sem encoding adequado
- Quando o número de clusters K é completamente desconhecido

```
from sklearn.cluster import KMeans from sklearn.preprocessing import StandardScaler # Pré-processamento scaler = StandardScaler() X_scaled = scaler.fit_transform(X) # Treino km = KMeans( n_clusters=4, init="k-means++", # inicializacao inteligente n_init=10, # 10 inicializacoes diferentes max_iter=300, random_state=42 ) labels = km.fit_predict(X_scaled) df["cluster"] = labels # Inertia para Elbow Method print(f"Inertia: {km.inertia_:.2f}")
```

6.2 K-Medoids (PAM — Partitioning Around Medoids)

Conceito: Similar ao K-Means, mas o representante de cada cluster é obrigatoriamente um ponto real do dataset (medoid), não uma média. Minimiza a soma das distâncias dos pontos ao seu medoid.

- **Vantagem sobre K-Means:** muito mais robusto a outliers — medoid não é arrastado por extremos.
- **Interpretabilidade:** o medoid é um cliente real — útil para apresentações executivas ("o cliente típico do segmento A tem estas características").
- **Distância:** aceita qualquer métrica de distância, inclusive Gower — ideal para dados mistos.
- **Desvantagem:** O(n^2) — mais lento que K-Means para datasets grandes.

```
from sklearn_extra.cluster import KMedoids # Com distância euclidiana (dados numéricos) kmed = KMedoids(n_clusters=4, metric="euclidean", init="k-medoids++", random_state=42) labels = kmed.fit_predict(X_scaled) # Com Gower Distance (dados mistos) import gower dist_gower = gower.gower_matrix(df_mixed) kmed_gower = KMedoids(n_clusters=4, metric="precomputed", random_state=42) labels = kmed_gower.fit_predict(dist_gower)
```

6.3 Clusterização Hierárquica (Agglomerative)

Conceito: Abordagem bottom-up. Começa com cada ponto como cluster individual e progressivamente une os clusters mais próximos até restar um único cluster. O resultado é uma hierarquia visualizada em dendrograma — K é escolhido cortando o dendrograma na altura desejada.

Linkage	Calcula distância entre clusters como...	Característica

Single	Mínima distância entre pontos dos clusters	Clusters alongados, sensível a ruído (chaining)
Complete	Máxima distância entre pontos dos clusters	Clusters compactos e esféricos
Average	Média das distâncias entre todos os pares	Equilíbrio entre single e complete
Ward	Minimiza a variância intra-cluster ao unir	Mais usado — resulta em clusters similares ao K-Means

Quando usar:

- Exploração inicial — entender hierarquia natural sem definir K
- Datasets pequenos a médios (< 10.000 linhas) — O(n² log n)
- Quando a relação hierárquica entre grupos tem significado de negócio
- Como validação cruzada com K-Means — se ambos chegam a grupos similares, são robustos

```
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster from sklearn.cluster import AgglomerativeClustering # Dendrograma para escolher K Z = linkage(X_scaled, method="ward") dendrogram(Z, truncate_mode="lastp", p=20, show_leaf_counts=True) # visualizar # Cortar em K clusters labels = fcluster(Z, t=4, criterion="maxclust") # Alternativa sklearn agg = AgglomerativeClustering(n_clusters=4, linkage="ward") labels = agg.fit_predict(X_scaled)
```

6.4 DBSCAN

Conceito: Density-Based Spatial Clustering of Applications with Noise. Agrupa pontos em regiões densamente conectadas. Não requer K — o número de clusters emerge dos dados. Detecta outliers nativamente como "noise points".

Definições fundamentais:

- **epsilon (eps):** raio da vizinhança de cada ponto.
- **min_samples:** número mínimo de pontos dentro do raio eps para ser core point.
- **Core point:** tem >= min_samples vizinhos dentro de eps.
- **Border point:** está dentro do eps de um core point, mas tem < min_samples vizinhos próprios.
- **Noise point:** nem core nem border — classificado como -1 (outlier).

Como escolher eps:

Para cada ponto, calcula a distância ao k-ésimo vizinho mais próximo (k = min_samples). Plota essas distâncias em ordem crescente. O "cotovelo" indica o eps adequado — distâncias abaixo do cotovelo = pontos densos (cluster), acima = ruído.

```
from sklearn.cluster import DBSCAN from sklearn.neighbors import NearestNeighbors import numpy as np # K-distance graph para escolher eps k = 5 # min_samples nn = NearestNeighbors(n_neighbors=k) nn.fit(X_scaled) dist, _ = nn.kneighbors(X_scaled) dist_k = np.sort(dist[:, k-1])[::-1] # distancia ao k-esimo vizinho # Plotar dist_k e identificar o cotovelo visualmente # Treino db = DBSCAN(eps=0.5, min_samples=5, metric="euclidean") labels = db.fit_predict(X_scaled) n_clusters = len(set(labels)) - (1 if -1 in labels else 0) n_noise = (labels == -1).sum() print(f"Clusters: {n_clusters}, Noise: {n_noise}")
```

Quando usar:

- Detecção de fraude e anomalias — noise points são o alvo
- Dados com clusters de forma irregular (não esféricos)
- Quando você não quer definir K a priori
- Dados geoespaciais com densidade variável

Limitação crítica: muito sensível a eps e min_samples. Com densidades muito diferentes no mesmo dataset, um único par de parâmetros não funciona bem para todos os clusters. HDBSCAN resolve essa limitação.

6.5 HDBSCAN — Hierarchical DBSCAN

Conceito: Versão hierárquica do DBSCAN. Lida com clusters de densidades diferentes — a principal limitação do DBSCAN. Mais robusto, menos sensível a parâmetros.

```
import hdbscan clusterer = hdbscan.HDBSCAN( min_cluster_size=50, # tamanho minimo de um cluster min_samples=5, # similar ao min_samples do DBSCAN cluster_selection_method="eom" # excess of mass ) labels = clusterer.fit_predict(X_scaled) # clusterer.probabilities_: probabilidade de pertencimento de cada ponto
```

6.6 Gaussian Mixture Models (GMM)

Conceito: Modelo probabilístico. Assume que os dados foram gerados por uma mistura de K distribuições gaussianas. Cada ponto tem uma PROBABILIDADE de pertencer a cada cluster (soft assignment) — diferente do K-Means que faz atribuição rígida (hard assignment).

- **Soft assignment:** P(cluster=k | ponto x) para cada k. Soma = 1.
- **Clusters elípticos:** GMM não assume esfericidade — pode modelar clusters alongados e rotacionados.
- **Covariance types:** spherical (esfera, similar K-Means), diag (eixos alinhados), tied (mesma forma), full (forma livre — mais flexível).
- **Seleção de K:** BIC e AIC — menor é melhor. Base estatística mais formal que Elbow.

```
from sklearn.mixture import GaussianMixture # Selecionar K via BIC bic_scores = [] for k in range(2, 11): gmm = GaussianMixture(n_components=k, covariance_type="full", random_state=42) gmm.fit(X_scaled) bic_scores.append(gmm.bic(X_scaled)) # Escolher K com menor BIC # Treino final gmm = GaussianMixture(n_components=4, covariance_type="full", random_state=42) gmm.fit(X_scaled) labels = gmm.predict(X_scaled) # hard assignment probs = gmm.predict_proba(X_scaled) # soft assignment df["cluster"] = labels df["prob_cluster0"] = probs[:, 0]
```

6.7 K-Modes e K-Prototypes

K-Modes: versão do K-Means para dados 100% categóricos. Usa moda ao invés de média e distância de Hamming (número de atributos diferentes).

K-Prototypes: combinação de K-Means (numéricas) e K-Modes (categóricas). Melhor opção nativa para dados mistos — sem necessidade de encoding manual.

```
from kmodes.kmodes import KModes from kmodes.kprototypes import KPrototypes # K-Modes (apenas categoricas) km_modes = KModes(n_clusters=4, init="Huang", n_init=5, random_state=42) labels = km_modes.fit_predict(df_cat) # K-Prototypes (misto) kp = KPrototypes(n_clusters=4, init="Cao", n_init=5, random_state=42) # categorical: lista de indices das colunas categoricas labels = kp.fit_predict(df.values, categorical=[2, 3, 5, 7])
```

Algoritmo	Tipo de Dado	Define K?	Robusto Outlier	Forma Cluster	Escala
K-Means	Numérico	Sim	Não	Esférica	Grande (O(nKd))
K-Medoids	Qualquer	Sim	Sim	Esférica	Médio (O(n^2))
Hierárquico	Qualquer	Não (dendrograma)	Moderado	Qualquer	Pequeno (O(n^2logn))
DBSCAN	Numérico	Não	Sim (detecta)	Qualquer	Grande
HDBSCAN	Numérico	Não	Sim	Qualquer	Grande
GMM	Numérico	Sim	Não	Elíptica	Médio
K-Modes	Categórico	Sim	Moderado	N/A	Grande
K-Prototypes	Misto	Sim	Moderado	Misto	Grande

7 · SELEÇÃO DO NÚMERO DE CLUSTERS

A escolha de K é a decisão mais crítica e mais subjetiva da clusterização. Não existe resposta matematicamente correta — existe o melhor K dado o objetivo de negócio. Use múltiplos métodos e triangule.

7.1 Elbow Method (Método do Cotovelo)

Plota a inércia (soma das distâncias quadráticas ao centroide) para cada K. A inércia sempre cai conforme K aumenta. O "cotovelo" — ponto onde a queda desacelera — indica o K ótimo.

Limitação: o cotovelo frequentemente não é óbvio. Nunca usar isolado.

```
inertias = [] K_range = range(2, 12) for k in K_range: km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42) km.fit(X_scaled) inertias.append(km.inertia_) # Plotar inertias vs K_range e identificar o cotovelo
```

7.2 Silhouette Score e Silhouette Plot

Silhouette Score médio: para cada ponto calcula:

- **a:** distância média aos outros pontos do mesmo cluster (coesão).
- **b:** distância média ao cluster vizinho mais próximo (separação).
- **silhouette = (b - a) / max(a, b)** — varia de -1 a 1.
- **Score médio:** média do silhouette de todos os pontos. Maior = melhor.

Silhouette Plot: visualiza o silhouette de cada ponto organizado por cluster. Clusters saudáveis têm silhouettes uniformes e positivos. Pontos com silhouette negativo estão mal alocados.

```
from sklearn.metrics import silhouette_score, silhouette_samples import numpy as np scores = [] for k in range(2, 12): km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42) labels = km.fit_predict(X_scaled) scores.append(silhouette_score(X_scaled, labels)) # Silhouette plot para K escolhido labels_best = KMeans(n_clusters=4, random_state=42).fit_predict(X_scaled) sil_vals = silhouette_samples(X_scaled, labels_best) # Plotar sil_vals agrupado por cluster
```

7.3 Davies-Bouldin Index

Razao entre a soma dos diametros dos clusters e a distancia entre os centroides. **Menor e melhor** — clusters compactos e bem separados.

```
from sklearn.metrics import davies_bouldin_score db_scores = [] for k in range(2, 12): labels = KMeans(n_clusters=k, random_state=42).fit_predict(X_scaled)
db_scores.append(davies_bouldin_score(X_scaled, labels))
```

7.4 Calinski-Harabasz Index

Razao entre a variancia inter-cluster e a variancia intra-cluster. **Maior e melhor** -- clusters densos e bem separados. Mais rapido de calcular que Silhouette.

```
from sklearn.metrics import calinski_harabasz_score ch_scores = [] for k in range(2, 12): labels = KMeans(n_clusters=k, random_state=42).fit_predict(X_scaled)
ch_scores.append(calinski_harabasz_score(X_scaled, labels))
```

7.5 BIC / AIC para GMM

BIC (Bayesian Information Criterion): penaliza complexidade do modelo. Menor BIC = melhor equilíbrio entre fit e complexidade. Critério mais rigoroso que AIC — preferível em produção.

AIC (Akaike Information Criterion): penaliza menos que BIC. Tende a escolher K maior. Útil quando se quer mais granularidade.

7.6 Dendrograma para Hierárquico

No algoritmo hierárquico, o dendrograma revela a hierarquia natural. Corte horizontal em alturas maiores = poucos clusters grandes. Corte em alturas menores = muitos clusters pequenos. Escolha o corte onde a distância entre uniões é maior — o "salto" maior indica K natural.

```
from scipy.cluster.hierarchy import dendrogram, linkage Z = linkage(X_scaled, method="ward")
plt.figure(figsize=(12, 6)) dendrogram(Z, truncate_mode="lastp", p=30, leaf_rotation=90,
leaf_font_size=8, show_contracted=True) plt.axhline(y=5, color="r", linestyle="--") # linha de corte
```

7.7 Gap Statistic

Compara a inércia do modelo com a inércia esperada em dados aleatórios uniformes. Gap(K) = E[log(Inércia_aleatória)] - log(Inércia_real). Escolher K com maior Gap — onde os dados têm mais estrutura que o aleatório. Mais robusto que Elbow, mais custoso computacionalmente.

Método	Direção	Base Matemática	Limitação
Elbow Method	Inércia mínima	Inércia (SSW)	Cotovelo raramente óbvio
Silhouette Score	Maximizar	Coesão e separação	Lento para n grande
Davies-Bouldin	Minimizar	Diâmetro / separação	Favorece clusters convexos
Calinski-Harabasz	Maximizar	Variância inter/intra	Favorece clusters grandes
BIC/AIC (GMM)	Minimizar	Verossimilhança penalizada	Apenas para GMM
Dendrograma	Visual	Hierarquia de distâncias	Subjetivo
Gap Statistic	Maximizar	Comparação com aleatório	Computacionalmente caro

Protocolo recomendado: sempre usar Elbow + Silhouette como dupla principal. Complementar com Davies-Bouldin. Consultar stakeholder de negócio sobre K operacionalmente viável. O K final é a interseção entre o que os dados sugerem e o que o negócio consegue operar.

8 · MÉTRICAS DE VALIDAÇÃO

Validação de clusterização opera em três camadas: métricas internas (sem ground truth), métricas externas (com labels de referência) e validação de negócio. As três precisam estar alinhadas.

8.1 Métricas Internas

Métrica	Fórmula Conceitual	Direção	Interpretação
Inércia (SSW)	Soma das dist² ao centroide	Minimizar	Não comparar entre K diferentes
Silhouette Score	(b-a)/max(a,b) média	Maximizar (max=1)	> 0.5 bom, > 0.7 excelente, < 0 ruim
Davies-Bouldin	Media(max(Si+Sj)/dij)	Minimizar	0 = perfeito — sem limite superior
Calinski-Harabasz	SSB/(K-1) / SSW/(n-K)	Maximizar	Sem limite — comparar relativamente
Dunn Index	min_dist_inter / max_diam_intra	Maximizar	Alto = clusters compactos e separados

```
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score def avaliar_clusters(X, labels): sil = silhouette_score(X, labels) dbi = davies_bouldin_score(X, labels) chi = calinski_harabasz_score(X, labels) print(f"Silhouette: {sil:.4f} (maior melhor)") print(f"Davies-Bouldin: {dbi:.4f} (menor melhor)") print(f"Calinski-Harabasz: {chi:.2f} (maior melhor)") return sil, dbi, chi
```

8.2 Métricas Externas

Quando há labels de referência — segmentos definidos manualmente pelo negócio, ou experimento com ground truth — compare com os clusters encontrados.

Métrica	Direção	Interpretação
Adjusted Rand Index (ARI)	Maximizar (max=1)	0=aleatório, 1=perfeito. Ajustado para chance — mais confiável que Rand Index
Normalized Mutual Info (NMI)	Maximizar (max=1)	Informação mútua normalizada entre clusters e labels
Fowlkes-Mallows Score	Maximizar (max=1)	Média geométrica de precisão e recall por par de pontos
Homogeneity	Maximizar	Cada cluster contém apenas membros de uma classe?
Completeness	Maximizar	Todos os membros de uma classe estão no mesmo cluster?
V-Measure	Maximizar	Média harmônica de homogeneity e completeness

```
from sklearn.metrics import (adjusted_rand_score, normalized_mutual_info_score, homogeneity_completeness_v_measure) ari = adjusted_rand_score(labels_true, labels_pred) nmi = normalized_mutual_info_score(labels_true, labels_pred) h, c, v = homogeneity_completeness_v_measure(labels_true, labels_pred) print(f"ARI: {ari:.4f} | NMI: {nmi:.4f} | V-Measure: {v:.4f}")
```

8.3 Estabilidade dos Clusters — Bootstrap

Clusters que mudam completamente a cada execução não são confiáveis. Teste de estabilidade via bootstrap: retreinar em amostras diferentes e medir similaridade entre partições com ARI.

```
from sklearn.utils import resample from sklearn.metrics import adjusted_rand_score aris = [] for _ in range(50): X_boot = resample(X_scaled, random_state=None) lab1 = KMeans(n_clusters=4, random_state=42).fit_predict(X_scaled) lab2 = KMeans(n_clusters=4, random_state=None).fit_predict(X_boot) # Nota: ARI entre partições diferentes requer alinhamento de labels aris.append(adjusted_rand_score(lab1[:len(X_boot)], lab2)) print(f"ARI medio: {np.mean(aris):.3f} +/- {np.std(aris):.3f}") # ARI > 0.8: clusters estáveis | < 0.6: instáveis
```

9 · VISUALIZAÇÕES E ANÁLISE DOS CLUSTERS

Visualização é onde clusterização ganha vida para o negócio. Métricas numéricas validam — visualizações explicam e convencem.

Visualização	O que revela	Quando usar
Scatter Plot 2D (PCA/UMAP)	Separação visual dos clusters no espaço reduzido	Sempre — validação visual básica
Silhouette Plot	Qualidade de alocação de cada ponto por cluster	Validação — identificar pontos mal alocados
Boxplot por Cluster	Distribuição de cada feature por cluster	Interpretação e nomeação dos clusters
Heatmap de Centroides	Valor médio de cada feature por cluster (Z-score)	Comparação rápida entre clusters
Parallel Coordinates	Perfil multidimensional de cada cluster	Apresentação executiva — padrões visuais
Dendrograma	Hierarquia natural dos dados	Hierárquico — escolha de K e estrutura
t-SNE / UMAP colorido	Separação em alta dimensão projetada em 2D	Após clusterização
Radar/Spider Chart	Perfil normalizado de cada cluster em múltiplas dimensões	Apresentação executiva — comparação
Treemap de tamanho	Proporção de clientes por cluster	Dashboard executivo
Bar Chart de distribuição	Feature categórica por cluster vs base total	Identificar características definidoras

9.1 Códigos Essenciais de Visualização

Heatmap de Centroides (Z-score):

```
import seaborn as sns import pandas as pd # Criar DataFrame de centroides centroids = pd.DataFrame(km.cluster_centers_, columns=feature_names) # Normalizar por Z-score para comparar features em escalas diferentes centroids_z = (centroids - centroids.mean()) / centroids.std() plt.figure(figsize=(12, 5)) sns.heatmap(centroids_z, annot=True, fmt=".2f", cmap="RdBu_r", center=0, linewidths=0.5) plt.title("Perfil dos Clusters — Z-score") plt.ylabel("Cluster") plt.xlabel("Feature")
```

Boxplot por Cluster:

```
df["cluster"] = labels features_plot = ["arpu", "consumo_dados", "reclamacoes", "tempo_contrato"] fig, axes = plt.subplots(2, 2, figsize=(12, 8)) for ax, feat in zip(axes.flatten(), features_plot): df.boxplot(column=feat, by="cluster", ax=ax) ax.set_title(feat) plt.suptitle("Distribuição das Features por Cluster") plt.tight_layout()
```

Parallel Coordinates:

```
from pandas.plotting import parallel_coordinates # Usar amostra se dataset grande df_sample = df.sample(500, random_state=42) cols = features_plot + ["cluster"] df_plot = df_sample[cols].copy() df_plot["cluster"] = df_plot["cluster"].astype(str) plt.figure(figsize=(14, 6)) parallel_coordinates(df_plot, "cluster", colormap="tab10", alpha=0.3) plt.title("Parallel Coordinates por Cluster")
```

UMAP + Scatter colorido:

```
import umap, matplotlib.pyplot as plt reducer = umap.UMAP(n_components=2, random_state=42) embedding = reducer.fit_transform(X_scaled) plt.figure(figsize=(10, 7)) scatter = plt.scatter(embedding[:,0], embedding[:,1], c=labels, cmap="tab10", s=5, alpha=0.6) plt.colorbar(scatter, label="Cluster") plt.title("UMAP — Projecao 2D dos Clusters")
```

9.2 Análise de Perfil — Como Nomear e Apresentar

Após validação matemática e visual, o trabalho real começa: interpretar o que cada cluster significa para o negócio.

• **Estatísticas descritivas por cluster:** média, mediana e desvio de cada feature. Identifica o que diferencia grupos.

• **Índice de penetração:** para categóricas, calcular proporção de cada categoria por cluster vs base total. Valores 2x acima da média = característica definidora.

• **Nomear com linguagem de negócio:** "Cluster 3" não é acionável. "Cliente de Alto Valor em Risco" orienta decisões.

• **Recomendar ação por cluster:** se dois clusters levam à mesma ação, considere fundi-los.

• **Tamanho de cada cluster:** clusters muito pequenos (< 1% da base) merecem investigação — podem ser outliers agrupados.

```
# Perfil completo por cluster perfil = df.groupby("cluster").agg({ "arpu": ["mean", "median", "std"], "consumo_dados": ["mean", "median"], "reclamacoes": ["mean", "sum"], "tempo_contrato": "mean", "tipo_plano": lambda x: x.value_counts().index[0] # moda }) tamanho = df["cluster"].value_counts(normalize=True).rename("proporcao") print(perfil) print(tamanho)
```

10 · PIPELINE COMPLETO EM PRODUÇÃO

10.1 Fluxo End-to-End

Etapa	Ação	Entregável
1. Objetivo	Definir unidade de análise, dimensões e ação planejada	Documento de escopo
2. Coleta e Reunir	Juntar todas as fontes relevantes com data de referência consistente	DataFrame base
3. EDA	Hopkins, distribuições, correlações, outliers, nulos	Relatório de qualidade
4. Pré-processamento	Imputação, capping, encoding, scaling	Pipeline sklearn persistido
5. Dim. Redução	PCA se > 15-20 features; FAMD se dados mistos	Componentes + variância explicada
6. Seleção de K	Elbow + Silhouette + Davies-Bouldin + consulta negócio	Justificativa de K
7. Treino	Múltiplos algoritmos — K-Means baseline + alternativas	Modelos treinados
8. Validação Matemática	Silhouette, DBI, CHI, estabilidade bootstrap	Tabela de métricas
9. Validação Visual	t-SNE/UMAP, boxplots, heatmap centroides	Conjunto de gráficos
10. Interpretação	Perfil por cluster, nomeação, índice de penetração	Apresentação executiva
11. Deploy	Atribuição de cluster a novos clientes (predict/transform)	Score de cluster em produção
12. Monitoramento	Estabilidade por cluster, distribuição das features	Dashboard de monitoramento

10.2 Código do Pipeline Completo

```
import pandas as pd import numpy as np from sklearn.preprocessing import StandardScaler from sklearn.decomposition import PCA from sklearn.cluster import KMeans from sklearn.metrics import silhouette_score, davies_bouldin_score from sklearn.pipeline import Pipeline import joblib # 1. PREPROCESSAMENTO scaler = StandardScaler() X_scaled = scaler.fit_transform(X) # 2. REDUCAO DE DIMENSIONALIDADE (se necessario) pca = PCA(n_components=0.85, random_state=42) # 85% variancia X_pca = pca.fit_transform(X_scaled) print(f"Componentes: {pca.n_components_}") # 3. SELECAO DE K results = [] for k in range(2, 11): km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42) labels = km.fit_predict(X_pca) results.append({ "k": k, "inertia": km.inertia_, "silhouette": silhouette_score(X_pca, labels), "dbi": davies_bouldin_score(X_pca, labels) }) df_results = pd.DataFrame(results) # 4. TREINO FINAL K_FINAL = 4 km_final = KMeans(n_clusters=K_FINAL, init="k-means++", n_init=10, random_state=42) km_final.fit(X_pca) df["cluster"] = km_final.labels_ # 5. PERSISTIR PIPELINE pipeline = {"scaler": scaler, "pca": pca, "kmeans": km_final} joblib.dump(pipeline, "clustering_pipeline.pkl") # 6. ATRIBUIR CLUSTER A NOVO DADO def predict_cluster(X_new, pipeline_path="clustering_pipeline.pkl"): pipe = joblib.load(pipeline_path) X_s = pipe["scaler"].transform(X_new) X_p = pipe["pca"].transform(X_s) return pipe["kmeans"].predict(X_p)
```

11 · MONITORAMENTO E RETREINAMENTO

Cluster em produção não é estático. A base de clientes evolui — novos produtos, campanhas, sazonalidade e mudanças macroeconômicas alteram o comportamento. Monitoramento contínuo é obrigatório.

11.1 PSI — Population Stability Index

PSI mede se a distribuição de uma variável mudou entre dois períodos. Aplicado em clusterização: monitorar se a proporção de clientes em cada cluster mudou.

PSI = soma[(% atual - % referencia) × ln(% atual / % referencia)]

PSI	Interpretação	Ação
< 0.10	Distribuição estável	Monitorar normalmente
0.10 - 0.20	Mudança moderada	Investigar causas
> 0.20	Mudança significativa	Retreinar modelo

```
def psi(expected, actual, buckets=10): """PSI entre duas distribuições de proporções de cluster""" def scale(arr): return arr / arr.sum() e = scale(np.array(expected) + 1e-6) # evitar log(0) a = scale(np.array(actual) + 1e-6) return np.sum((a - e) * np.log(a / e)) # Proporcao por cluster: referencia vs atual ref = df_ref["cluster"].value_counts(normalize=True).sort_index() cur = df_cur["cluster"].value_counts(normalize=True).sort_index() print(f"PSI clusters: {psi(ref, cur):.4f}")
```

11.2 Drift das Features

• **Distribuição das features de entrada:** calcular PSI de cada feature entre período de treino e produção atual. PSI > 0.2 em features críticas = sinal de retreinamento.

• **Variância e média por cluster ao longo do tempo:** centroides estáveis = modelo estável. Centroide que migra = perfil do cluster mudou.

• **Proporção de clusters ao longo do tempo:** cluster que dobrou de tamanho em 2 meses merece investigação — pode ser efeito de campanha ou mudança de comportamento.

11.3 Protocolo de Retreinamento

• **Trigger por PSI:** PSI > 0.2 em cluster ou feature crítica — retreinar.

• **Trigger por tempo:** retreinamento preventivo semestral (base estável) ou trimestral (base dinâmica).

• **Trigger por evento de negócio:** novo produto, mudança de portfólio, fusão, campanha massiva — retreinar.

• **Validar antes de substituir:** novo modelo precisa ter métricas iguais ou melhores que o anterior. ARI entre novo e antigo para entender quanto mudou.

• **Versionar modelos:** manter histórico de modelos com data, métricas e snapshot dos dados de treino.

12 · CHECKLIST DO ESPECIALISTA

Use antes de apresentar qualquer resultado de clusterização. Item não marcado = risco de cluster inválido ou inútil.

	Item	Etapa
■	Objetivo de negócio definido: o que será feito com os clusters?	Problema
■	Unidade de análise correta (cliente, contrato, transação)?	Problema
■	K operacionalmente viável discutido com stakeholder?	Problema
■	Features selecionadas por dimensão de negócio, não disponibilidade?	Features
■	Variáveis de ID e proxies do target removidas?	Features
■	Nulos imputados (zero nulos no dataset de entrada)?	Pré-proc.
■	Outliers tratados (capping ou remoção justificada)?	Pré-proc.
■	Features escaladas (StandardScaler ou RobustScaler)?	Pré-proc.
■	Variância zero e quasi-constantes removidas?	Pré-proc.
■	Hopkins Statistic calculado (estrutura confirmada)?	EDA
■	Correlações e multicolinearidade analisadas?	EDA
■	t-SNE ou UMAP plotado para visualizar estrutura?	EDA
■	PCA/FAMD aplicado se > 15 features?	Dim. Red.
■	Variância acumulada do PCA documentada?	Dim. Red.
■	Elbow + Silhouette + DBI calculados para K=2..10?	Seleção K
■	K final justificado por critério matemático E de negócio?	Seleção K
■	Múltiplos algoritmos testados (K-Means + alternativa)?	Algoritmo
■	n_init >= 10 usado no K-Means?	Algoritmo
■	Silhouette Score documentado para o modelo final?	Validação
■	Davies-Bouldin calculado?	Validação
■	Teste de estabilidade bootstrap com ARI realizado?	Validação
■	t-SNE/UMAP colorido por cluster gerado?	Visualização
■	Heatmap de centroides (Z-score) gerado?	Visualização
■	Boxplot de features por cluster gerado?	Visualização
■	Parallel Coordinates gerado?	Visualização
■	Cada cluster tem nome de negócio (não "Cluster 3")?	Interpretação
■	Ação diferente definida para cada cluster?	Interpretação
■	Clusters muito pequenos (< 1%) investigados?	Interpretação
■	Pipeline persistido (scaler + pca + modelo)?	Produção
■	PSI de monitoramento configurado?	Monitoramento
■	Protocolo de retreinamento documentado?	Monitoramento